had an attribute 'article' set to the article to display, in this template we are accessing the details from that article object.

Now let's go over the syntax of Django's template language. Also remember that it's possible to write your own code that extends Django's template syntax.

## Displaying Data

Displaying stuff in the right place is one of the most important things you probably want to do in a template, and the syntax for that is quite simple, as you saw in the previous example. All you need to do to print a variable is to enclose the name of the variable inside double curly braces. As you saw in the previous example, you can access the attributes of an object as well, however it is also possible to access elements of an array as follows:

```
{{ array.1 }}
```

Essentially for anything after the period it will check to see if it is an attribute or function or array index. In case it is a function it will call the function (if it supports being called without any arguments) and print the value.

## Conditionals

There are cases where you might want to display different messages based on some or the other condition, for that we have the `if` block that can be used as follows:

```
{% if article.publish _ status %}
    Published
{% else %}
    Not Published
{% endif %}
```

How this works should be pretty obvious given the above example. Since publish_status is already a binary field this works out for us, however this can also be useful to check if a property has been set. For instance since our featured image is optional, we can use code like the above to check if a featured image is set, and to hide the image block if there is no image to display.

The 'if' block also also obviously evaluate conditionals like the following:

```
{% if article.tags.count > 2 %}
```

The article has more than two tags.

```
{% endif %}
```

## Loops

Speaking of tags, articles have many of those, so we need to be able to display